

AEI-V2G PROJECT

Full Context Document for Claude Code / VS Code

ISAC-Aided Proactive Vehicle-to-Grid Scheduling with Blockchain Incentive Layer

Version: May 2026 | Status: Active Research + Development

Prepared for handoff to Claude Code in VS Code

1. How to Use This Document

This document is the complete project briefing for Claude Code. It contains everything needed to understand, extend, and improve the AEI-V2G codebase without any prior conversation history. Read it fully before touching any file. Sections are ordered from high-level to low-level: research context first, then system architecture, then what has been built, then what needs to be built next.

IMPORTANT: This is a research project with an academic paper as its primary deliverable. Every code change must be traceable to a research objective. Do not refactor for style alone. Do not introduce new dependencies without checking they are available on Raspberry Pi (ARM, Python 3.11).

2. Project Overview

2.1 What This Project Is

AEI-V2G (Adaptive Edge Intelligence for Vehicle-to-Grid) is a distributed edge computing system that uses Integrated Sensing and Communication (ISAC) technology to enable proactive, sensing-driven Vehicle-to-Grid (V2G) energy scheduling. The system runs on a cluster of Raspberry Pi nodes, communicates via MQTT, and records all decisions to a private Ethereum blockchain.

The project has two parallel tracks that must converge:

- Track 1 — Research paper: A formal academic contribution showing that ISAC sensing improves V2G scheduling over reactive baselines. This requires a simulation/evaluation layer that computes standard research metrics (PAR, TEC, SoC satisfaction, scheduling lag).
- Track 2 — Hardware prototype: The live Raspberry Pi cluster (AEI-V2G) that demonstrates physical deployment of the system. This validates the architecture but is not yet wired to the research metrics.

2.2 The Core Research Problem

Current V2G scheduling systems are reactive — they only know about an EV after it physically plugs into the charger. This means the grid operator has zero advance notice, cannot pre-allocate power, and cannot smooth grid demand peaks before they occur.

Our proposed solution: deploy ISAC-equipped Roadside Units (RSUs) along the approach corridor. These RSUs simultaneously transmit communication signals AND sense approaching vehicles using the reflected echo — extracting speed, distance, and estimated time of arrival (ETA) for each EV before it arrives. This sensing data feeds a scheduling algorithm that commits a charging/discharging plan proactively, before plug-in.

The claim we must prove with results: ISAC-aided proactive scheduling reduces peak-to-average ratio, total energy cost, and scheduling lag compared to reactive V2G baselines.

2.3 Why This Is Novel

- ISAC literature (including the anchor paper IBTD, IET Comms 2024) applies sensing to communication metrics — beam tracking, packet age. No paper applies ISAC sensing to energy dispatch.
- V2G literature assumes EV arrival distributions are known statistically. No paper uses physical real-time sensing of approaching EVs.
- Our system combines ISAC + V2G + Blockchain incentive layer. As of May 2026, no published paper covers all three together.
- The pre-arrival slot reservation action — committing a dispatch plan before plug-in — is a new action type not present in any existing V2G scheduler.

3. Research Objectives (Formal)

These are the agreed research objectives. All development work must map to at least one of these.

- Obj-1: Investigate and model the integration of ISAC technology with V2G energy management systems.
- Obj-2: Use ISAC-equipped RSUs to detect approaching EVs and extract speed, distance, and ETA before physical plug-in.
- Obj-3: Develop a proactive scheduling framework that prepares a charge/discharge plan before EV arrival, replacing reactive plug-in-triggered scheduling.
- Obj-4: Incorporate per-EV battery state (SoC, SoH) communicated from the EV to the RSU, enabling personalised energy dispatch decisions.
- Obj-5: Design a blockchain-backed credit-point incentive mechanism that rewards EV owners for discharging during grid stress, with points redeemable for future charging credits.
- Obj-6: Use blockchain for secure, transparent, tamper-proof storage of all energy transactions and credit point balances.
- Obj-7: Optimise the schedule to reduce peak grid load (PAR), minimise EV user electricity cost (TEC), and preserve battery health (degradation cost).
- Obj-8: Apply a deep reinforcement learning-based scheduling algorithm that learns an optimal dispatch policy under uncertainty in EV arrivals, user behaviour, and grid conditions.
- Obj-9: Evaluate against reference baselines using: PAR, TEC, battery degradation cost, SoC satisfaction ratio, scheduling lag, sensing gain ΔJ .
- Obj-10: Demonstrate that sensing-aided proactive scheduling outperforms reactive V2G, particularly under high EV density and time-varying grid demand.

Note on Obj-8: The scheduling algorithm is NOT yet finalised. DRQN (Deep Recurrent Q-Network with LSTM) is the leading candidate because of its alignment with the IBTD anchor paper architecture. However, do not hard-code DRQN-specific assumptions into shared modules. The scheduler must be pluggable.

4. System Architecture

4.1 Node Roles (Physical Raspberry Pi Cluster)

The system runs on five Raspberry Pi nodes connected over Tailscale VPN. Each node has a single defined role. MQTT (broker on pi1 port 1883) is the messaging bus between all nodes.

Node	Role	Tailscale IP	Responsibility
pi1	LAVA validator	100.112.171.127	Runs the scheduling decision engine (LAVA). Publishes route and V2G dispatch decisions. Hosts MQTT broker.
pi2	Station validator	100.99.112.11	Manages station_a (km 8) and station_b (km 24). Receives route decisions, admits EVs, applies V2G commands, tracks per-station state.
pi3	Station validator	100.94.93.69	Manages station_c (km 41). Same role as pi2 but for the furthest station. Was idle in the 20-min run — needs investigation.
pi5	RSU observer	100.84.220.4	Generates and senses EV arrival features. Publishes to aei/rsu/sense. This is the ISAC sensing layer. 78 observations in 20 ticks.
pi6	Grid observer	100.79.78.93	Replays the CAISO real grid demand profile. Publishes grid stress state. Average stress 0.8246 in the last run.

4.2 MQTT Message Topics

Topic	Producer	Content and Purpose
aei/rsu/sense	pi5	EV arrival features: ev_id, distance_km, speed_kph, eta_seconds, estimated_soc (proxy). Published once per tick per EV in RSU range.
aei/grid/state	pi6	Current grid stress (0-1 normalised CAISO load), TOU price, peak/off-peak flag. Published every tick.
aei/lava/route	pi1	EV-to-station routing decision: ev_id, assigned_station, ev_feature snapshot (SoC, battery capacity, speed), ETA at station.
aei/lava/v2g	pi1	V2G dispatch command: station_id, power_kw (positive = charge, negative = discharge), grid_stress at time of decision.
aei/station/state	pi2, pi3	Station queue snapshot: active EVs, queued EVs, current load kW, available slots. Used by pi1 for load-aware routing.
aei/chain/sync	all	Local JSONL hash-chain tail broadcast. Used for cross-node audit verification.

4.3 Physical Corridor Configuration

The simulation models a 50 km highway corridor with four RSUs and three charging stations.

- Corridor length: 50 km. EV arrivals: 28 per hour. Tick size: 60 seconds.
- RSU positions: rsu_1 at km 4, rsu_2 at km 16, rsu_3 at km 31, rsu_4 at km 45. RSU sensing range: 7.5 km each.
- Station A: km 8, max 360 kW, 6 slots, base price \$0.31/kWh.
- Station B: km 24, max 420 kW, 7 slots, base price \$0.29/kWh.
- Station C: km 41, max 300 kW, 5 slots, base price \$0.34/kWh.
- Grid: CAISO 2024-05-01 to 2024-05-07 real demand profile. Base stress 0.45. Peak hours: 07, 08, 17, 18, 19. Stress event at hour 18 with +0.22 boost. V2G base price: \$0.42/kWh. Critical grid stress threshold: 0.8.

5. The LAVA Scheduling Engine — Current State and Required Evolution

5.1 What LAVA Is Now

LAVA (the current scheduling engine on pi1) is a weighted scoring heuristic with a rules engine and constraints checker. It is NOT a deep reinforcement learning algorithm. When making a routing decision, it scores each station using a weighted sum of four factors:

- Grid stress weight: 0.32 — penalises routing to stations in high-stress grid zones.
- EV wait time weight: 0.28 — penalises routing that would increase queue wait.
- Station imbalance weight: 0.22 — penalises over-concentrating EVs at one station.
- Energy cost weight: 0.18 — prefers lower-price stations.

V2G dispatch is triggered by a simple threshold rule: if `grid_stress > 0.8` (`critical_grid_stress`), dispatch V2G at a fixed power level. The system invited 10 EVs, 5 accepted, 1 kWh supplied, \$0.60 revenue in the 20-minute run.

Decision latency is sub-millisecond (avg 0.397 ms) because the decision is a fast weighted sum, not a neural network inference pass.

5.2 What LAVA Must Become

To satisfy Obj-8 and produce publishable results, LAVA must be upgraded from a heuristic to a learning-based scheduler. The exact algorithm is not yet finalised. The leading candidate is DRQN (Deep Recurrent Q-Network with LSTM), chosen because:

- The IBTD anchor paper (which our paper directly adapts) uses DRQN + LSTM. Architectural alignment strengthens the paper's positioning.
- The V2G problem is a POMDP (partially observable Markov decision process) — the agent cannot perfectly know future EV arrivals or battery states. LSTM handles this by carrying temporal memory across time steps.
- DRQN uses a discrete action space, which maps naturally to per-EV decisions: {charge at slow/normal/fast rate, discharge at slow/normal/fast rate, idle}.

CRITICAL DESIGN CONSTRAINT: The scheduler module must be implemented as a pluggable class with a standard interface (`observe(state) -> action`, `train(experience) -> loss`). LAVA heuristic and any DRL algorithm must be swappable without changing the coordinator, RSU, or station code. This is essential for the ablation study (compare ISAC+DRL vs. ISAC+heuristic vs. no-ISAC+DRL).

5.3 DRQN State Vector (Proposed)

The state vector fed to the DRQN at each time slot t must include inputs from both the ISAC sensing layer and the grid/EV state. This is the full proposed state:

Input group	Variable	Description	Source
ISAC sensing	<code>d_k</code>	Distance of approaching EV k from station (km)	pi5 / aei/rsu/sense

ISAC sensing	v_k	Speed of approaching EV k (km/h)	pi5 / aei/rsu/sense
ISAC sensing	$t_{\hat{arr}_k}$	Predicted arrival time = d_k / v_k (seconds)	Derived from sensing
ISAC sensing	$SoC_{\hat{at}_k}$	Pre-plug-in SoC proxy (radar/channel inference)	pi5 — TO BE IMPLEMENTED
EV on-board	$SoC_k(t)$	True battery state of charge (%) at time t	EV -> RSU -> pi1
EV on-board	SoH_k	Battery state of health — capacity fade factor (0-1)	EV -> RSU -> pi1
EV on-board	SoC_{req_k}	Driver's requested SoC at departure	EV -> RSU -> pi1
EV on-board	T_{dep_k}	Declared departure time	EV -> RSU -> pi1
Grid	$P_{grid}(t)$	Current aggregate grid load (kW)	pi6 / aei/grid/state
Grid	$\lambda(t)$	Real-time TOU electricity price (\$/kWh)	pi6 / aei/grid/state
Grid	$\lambda_{forecast}$	Price forecast for next H time slots	pi6 / derived
Time	t	Current time slot index	System clock

5.4 DRQN Action Space (Proposed)

For each plugged-in EV k at time slot t , the agent outputs:

- Mode: a_k in {charge, discharge, idle}
- Power level: P_k in { P_{slow} (3.3 kW), P_{normal} (7.2 kW), P_{fast} (11 kW)}
- Pre-arrival reservation (novel): for detected approaching EVs, reserve a charger slot $t_{\hat{arr}_k} - \delta$ slots ahead. This action does not exist in any prior V2G paper and is the primary novelty of the scheduling layer.

5.5 Reward Function (Proposed)

$r_t = -\alpha * \delta_{PAR}(t) - \beta * TEC(t) - \gamma * C_{deg}(t) + \delta * SoC_{sat}(t) - \epsilon * scheduling_lag(t)$

- $\delta_{PAR}(t)$: increase in peak-to-average ratio this slot. Penalised to encourage peak shaving.
- $TEC(t)$: electricity cost this slot = power * TOU_price * slot_duration.
- $C_{deg}(t)$: battery degradation cost using weighted Ah-throughput (wAh) model.
- $SoC_{sat}(t)$: positive reward for EVs that reach SoC_{req_k} by T_{dep_k} . Also a hard terminal penalty if violated.
- $scheduling_lag(t)$: time from EV arrival to first dispatch decision. Our system should be near zero (pre-arrival); reactive baselines will be 30-90 seconds.
- Tunable weights α , β , γ , δ , ϵ to be determined during training experiments.

6. Blockchain and Incentive Layer

6.1 Current Blockchain Implementation

A private Ethereum chain (Hardhat JSON-RPC on Render) is already operational. Every decision made by every node is SHA-256 hashed and the hash is sent as calldata in a 0-ETH transaction to a sink account. This provides tamper-evident audit logging.

Chain URL: <https://myprivatechain.onrender.com>

Chain ID: Hardhat 0x539

Log sink account: 0x90F79bf6EB2c4f870365E785982E1f101E93b906

Key source file: `logging_layer/chain_client.py`

After the 20-minute clean run, the chain reached block 0x98 (152 blocks). Each node produced an append-only JSONL hash-chain log. Chain records: pi1=27, pi2=46, pi3=39, pi5=20, pi6=20.

6.2 What the Blockchain Must Become

The current blockchain records hashes for audit. It does NOT yet implement the credit-point incentive mechanism. This is a required addition (Obj-5, Obj-6). The following needs to be designed and built:

- Smart contract for credit point ledger: each EV wallet address has a credit balance. Discharging to grid during a peak window awards N credits per kWh supplied. Credits are redeemable for discounted future charging.
- V2G settlement record on-chain: when an EV completes a V2G discharge session, the smart contract records: `ev_id`, `station_id`, `kWh_supplied`, `credits_awarded`, `timestamp`.
- Credit redemption: when an EV requests charging, it can submit a credit redemption transaction. The smart contract deducts credits and issues a charging discount.
- Points-for-discharge rate: the current system already has `v2g_base_price = $0.42/kWh` as a monetary incentive. The credit system is additive — EVs earn both cash and points. The points rate needs to be defined (suggested: 1 credit per 0.5 kWh discharged during peak).

The credit-point smart contract is a new Solidity file. The existing `chain_client.py` will need new methods: `award_credits(ev_id, kwh)`, `redeem_credits(ev_id, amount)`, `get_balance(ev_id)`. The Hardhat development environment is already set up.

7. Performance Metrics — What Must Be Measured

These are the metrics the paper must report. Currently NONE of them are computed by the live system. A metrics computation module must be added as a post-processing step over the JSONL logs, or computed in real-time during simulation runs.

7.1 Primary Metrics (Must Appear in Every Result Figure)

Metric	Formula / Unit	Justification and Target
Peak-to-Average Ratio (PAR)	$P_{\text{peak}} / P_{\text{avg}}$ over episode	Primary grid-side metric. Target: our system achieves lower PAR than all 4 baselines. Benchmark: Paper 5 achieves 97.37% load variance reduction.
Total Energy Cost (TEC)	$\text{Sum}(P_k * \lambda(t) * dt)$ per episode (\$)	Primary user-side metric. Target: meaningful cost reduction vs. uncontrolled charging and reactive V2G.
SoC Satisfaction Ratio	% EVs reaching $\text{SoC}_{\text{req}_k}$ by T_{dep_k}	User comfort constraint. Must remain above 90% or the system is not practically deployable.
Scheduling Lag	Time from EV arrival to first dispatch decision (ms)	This is where our system shines. Reactive systems: 30-90 seconds. ISAC-aided system: near-zero (pre-arrival reservation).
Sensing Gain (delta-J)	$\text{Reward}_{\text{ISAC}} - \text{Reward}_{\text{no_ISAC}}$	Marginal improvement attributable purely to ISAC sensing. This is the core contribution metric.

7.2 Secondary Metrics

Metric	Formula / Unit	Purpose
Battery Degradation Cost	wAh model: $\text{sum}(\text{SoC}_{\text{change}} * C_{\text{deg_rate}})$	Validates that V2G cycling does not excessively wear batteries. Keep degradation increase below 15% over baseline.
SoC Proxy Estimation Error	MAE: $ \text{SoC}_{\text{hat}_k} - \text{SoC}_{\text{true}_k} $ at plug-in	Validates the ISAC pre-plug-in SoC inference model. Novel contribution — no prior paper reports this.
Detection Probability (P_d)	% EVs correctly detected before arrival	Validates RSU sensing coverage and the EV detection pipeline.
Cumulative Reward Convergence	Episode reward over training steps	Validates DRL training stability and that ISAC-informed state converges faster than no-sensing state.
Decision Latency	ms per scheduling decision (hardware)	Already measured: 0.397 ms avg. Continue tracking as scheduler complexity grows.

8. Baseline Comparison Plan

The paper must compare our proposed system against at least 3 baselines. All baselines must run on the same simulation environment to ensure fair comparison.

Baseline	Name	Algorithm	Key difference from ours
B1	Reactive V2G (no sensing)	Current LAVA heuristic	No sensing at all. Scheduler only acts after plug-in. Scheduling lag = full reaction delay.
B2	Statistical arrival model	DRL with Gaussian arrival distribution	Uses statistical EV arrival priors instead of real-time ISAC sensing. No SoC proxy.
B3	LAVA heuristic + ISAC sensing	Current LAVA with sensing inputs added	Has sensing but uses rule-based weights instead of learned DRL policy. Shows the value of learning.
Ours	ISAC-DRL-V2G (proposed)	DRL (TBD) + ISAC sensing + blockchain	Full system: proactive sensing + learned policy + blockchain incentive + pre-arrival reservation.

9. What Has Been Built — Current Codebase State

9.1 What Is Working

- Five-node Raspberry Pi deployment over Tailscale VPN — verified operational.
- MQTT messaging between all nodes using Mosquitto broker on pi1.
- RSU sensing pipeline (pi5): EV arrival features published to aei/rsu/sense each tick. 78 observations in 20 ticks.
- Grid observer (pi6): CAISO real demand profile replay with configurable start minute (AEI_GRID_START_MINUTE). Average stress 0.8246 in last run.
- LAVA routing decisions (pi1): 7 unique EVs routed in 20 ticks at 0.397 ms average latency.
- Station-side EV admission (pi2): receives route decisions, schedules ETA, materialises EVs into charging slots. 2 EVs arrived and actively charging at run end.
- V2G dispatch pipeline: pi1 dispatches 36 kW V2G when stress > 0.8. pi2 applies to active EVs. 5 accepted, 1 kWh supplied, \$0.60 revenue.
- Blockchain audit layer: SHA-256 hash of every decision anchored to private Ethereum chain. Block 0 to 0x98 in 20 minutes.
- Per-node append-only JSONL audit logs with hash chaining.

9.2 What Is Missing or Incomplete

- SoC and SoH per-EV tracking: currently only a minimum battery threshold constraint exists (v2g_min_battery = 55, min_battery_after_v2g = 20). Full SoC trajectory tracking per EV is not implemented.
- SoC proxy inference from ISAC sensing: pi5 generates EV features but does not currently output an SoC estimate from radar/channel data. This is the most novel technical contribution and does not yet exist in the codebase.
- DRL scheduler: the learning-based scheduling algorithm has not been implemented. LAVA is a weighted heuristic. The DRL module needs to be built as a pluggable replacement.
- Credit-point incentive smart contract: the blockchain records hashes but has no EV credit ledger, no V2G reward minting, and no credit redemption logic.
- Metrics computation module: PAR, TEC, SoC satisfaction ratio, scheduling lag, and sensing gain delta-J are not computed anywhere in the codebase.
- Pre-arrival slot reservation action: LAVA routes EVs to stations but does not reserve charger slots before arrival. This new action type must be added.
- Station C underutilisation: pi3 received zero routes in the 20-minute run. The routing logic may be biased toward pi2 stations — needs investigation.
- Simulation mode: the system runs in real-time hardware mode only. A fast simulation mode (no sleep, replay at 10x-100x speed) is needed to run the DRL training experiments.

10. Key File Paths and Module Responsibilities

Path	Responsibility
integration/coordinator.py	Main orchestrator. Starts all roles based on node identity. Entry point for cluster launch.
logging_layer/chain_client.py	Blockchain interface. SHA-256 hashing and on-chain anchoring. Must be extended with credit_ledger contract calls.
deploy/_launch_cluster.ps1	PowerShell script to SSH into all Pis and launch the coordinator with correct role and duration.
deploy/aggregate_results.ps1	Pulls piN_metrics.json from all nodes and merges into cluster_metrics.json.
deploy/_run_venv_setup.ps1	Sets up Python venv on each Pi with all dependencies.
data/grid_profiles/caiso_*.csv	Real CAISO demand profiles used by pi6 grid observer.
reports/cluster_metrics.json	Aggregated run results. The file analysed in the alignment report.
reports/AEI_V2G_DISTRIBUTED_CLUSTER_RUN.md	Human-readable run journal. Contains reviewer-risk wording and figure plans.

Files That Need to Be Created (Not Yet Existing)

Suggested path	Purpose
scheduling/base_scheduler.py	Abstract base class defining the scheduler interface: observe(state) -> action, train(experience), save(), load(). Both LAVA and DRL must implement this interface.
scheduling/lava_scheduler.py	Refactored LAVA heuristic implementing the base interface. Must be extractable from coordinator.py without breaking existing runs.
scheduling/drl_scheduler.py	DRL-based scheduler implementing the base interface. Algorithm TBD. Pluggable replacement for LAVA.
sensing/soc_proxy.py	SoC proxy inference model. Input: ISAC echo features (RSSI, Doppler, range). Output: estimated SoC_hat_k. This is the most novel technical component.
metrics/evaluator.py	Computes PAR, TEC, SoC satisfaction ratio, scheduling lag, degradation cost, sensing gain from episode logs.
simulation/fast_sim.py	Headless simulation mode for DRL training. Replays CAISO profile at configurable speed. No Raspberry Pi hardware required.
contracts/CreditLedger.sol	Solidity smart contract for EV credit point system. Functions: award_credits, redeem_credits, get_balance, get_transaction_history.

evaluation/baselines.py	Runs all baselines (B1, B2, B3) under the same simulation conditions and records metrics for comparison.
-------------------------	--

11. Reference Papers and How They Relate to the Code

#	Paper	How it relates to the codebase
1	IBTD — IET Comms 2024 (Zhao et al.)	Anchor paper. We adapt its DRQN+LSTM+ISAC architecture. The sensing pipeline on pi5 and the DRQN in scheduling/drl_scheduler.py directly mirror IBTD's design, with state/action/reward redefined for V2G energy dispatch instead of beam tracking.
2	Blockchain Energy Trading — Energies 2022	Reference for the credit-point smart contract design. The double auction and credit scoring model in CreditLedger.sol should follow this paper's architecture.
3	Transfer DRL V2G — arXiv 2022	Strongest performance benchmark. Our system must beat its load variance and cost reduction figures. Use its state space design as reference for the DRL state vector.
4	RPPO V2G — World EV Journal 2024 (He et al.)	Closest methodological match. Also uses LSTM-based recurrent DRL for bidirectional V2G with TOU pricing. The RPPO algorithm is an alternative to DRQN if the algorithm choice changes.

12. Development Priorities — Ordered Task List

Work in this order. Do not move to a later task until the earlier one is complete and tested.

Priority 1 — Pluggable Scheduler Interface (scheduling/base_scheduler.py)

Refactor the coordinator so the scheduler is injectable. Extract LAVA into `lava_scheduler.py`. Write a unit test that swaps LAVA for a dummy random scheduler without any other code changing. This unblocks all further DRL work.

Priority 2 — Per-EV SoC Tracking (station-side)

Add `SoC_k(t)`, `SoH_k`, `SoC_req_k`, and `T_dep_k` to the EV feature data published on `aei/rsu/sense` and stored in station state. Station validators must track SoC trajectory for each admitted EV across ticks. This is required for the reward function and for the metrics module.

Priority 3 — Metrics Computation Module (metrics/evaluator.py)

Build the evaluator that processes episode JSONL logs and outputs: PAR, TEC, SoC satisfaction ratio, scheduling lag, battery degradation cost. Run it against the existing 20-minute `cluster_metrics.json` to validate it produces sensible numbers before any DRL work begins.

Priority 4 — Fast Simulation Mode (simulation/fast_sim.py)

Build a headless simulation that replays the CAISO profile and generates synthetic EV arrivals at configurable speed (1x, 10x, 100x real time). This must produce the same MQTT message structure as the real hardware cluster so the scheduler and metrics modules work identically in simulation and on hardware. DRL training requires thousands of episodes — this is essential.

Priority 5 — DRL Scheduler (scheduling/drl_scheduler.py)

Implement the learning-based scheduler using the base interface. Use the state vector and reward function from Section 5. Train using the fast simulation mode. Compare convergence and performance against the LAVA heuristic baseline using the metrics module.

Priority 6 — SoC Proxy Inference (sensing/soc_proxy.py)

Build the SoC estimation model that infers battery state from ISAC sensing features before plug-in. This is the most technically novel component. Start with a simple regression model (ridge or random forest) trained on synthetic (`sensing_features`, `true_SoC`) pairs. The input features are: RSSI of echo, Doppler shift, range, vehicle deceleration rate. Report MAE as the sensing accuracy metric.

Priority 7 — Credit-Point Smart Contract (contracts/CreditLedger.sol)

Write the Solidity contract for the credit-point incentive system. Deploy to the existing private Hardhat chain. Update `chain_client.py` with `award_credits`, `redeem_credits`, `get_balance`

methods. Wire V2G settlement records to trigger credit awards automatically when V2G discharge is applied by the station validator.

Priority 8 — Baseline Evaluation (evaluation/baselines.py)

Run all three baselines (B1 reactive, B2 statistical, B3 LAVA+ISAC) and the proposed system through the same simulation episodes. Collect all metrics. Produce the comparison tables and figures for the paper.

13. Hard Constraints and Coding Rules

- All Python must run on Raspberry Pi 4 (ARM64, Python 3.11). Test imports for ARM compatibility before adding new packages (e.g. PyTorch has ARM wheels; TensorFlow does not).
- The MQTT message schema is fixed. Do not change topic names or field names without updating all producers and consumers simultaneously and re-testing the cluster.
- The blockchain RPC endpoint is a free-tier Render service and may cold-start slowly. `chain_client.py` must handle connection timeouts gracefully with retry logic.
- Do not delete or modify the existing JSONL chain logs or `cluster_metrics.json`. These are research artifacts. Write new outputs to new files.
- The scheduler must remain pluggable. Never hard-code LAVA logic into the coordinator. Never hard-code DRL assumptions into the station validators.
- Battery SoC constraints are safety-critical: SoC must never fall below `min_battery_after_v2g = 20%` due to scheduling decisions. Add a safety layer that overrides any scheduler action violating this.
- When in doubt about the research direction, refer back to the 10 objectives in Section 3. Each code change should map to at least one objective.
- Station C (pi3) received zero routes in the 20-minute run. Do not assume it is broken before investigating. Check the LAVA routing weights and ETA calculations for km 41 corridor.

14. Glossary of Key Terms

Term	Definition
ISAC	Integrated Sensing and Communication. A wireless technology that uses a single transmitted signal to both communicate data and sense the physical environment via echo returns. The same waveform does two jobs.
V2G	Vehicle-to-Grid. Bidirectional power flow between an electric vehicle battery and the electricity grid. EVs can discharge stored energy back to the grid when demand is high.
V2B	Vehicle-to-Building. Same as V2G but EV discharges to support a specific building's load rather than the wider grid.
RSU	Roadside Unit. A fixed network node at the side of the road that communicates with vehicles and, in our system, also senses them using ISAC echo signals.
LAVA	The current scheduling engine on pi1. A weighted scoring heuristic with a rules engine and constraints checker. NOT a deep learning algorithm. Must be replaced by or supplemented with a DRL scheduler.
DRQN	Deep Recurrent Q-Network. DQN (Deep Q-Network) augmented with an LSTM layer that carries hidden state across time steps. Handles partial observability in sequential decision problems. Leading candidate for the scheduling algorithm.
POMDP	Partially Observable Markov Decision Process. The formal model for our scheduling problem. The agent (scheduler) cannot perfectly observe the true state (exact SoC, exact arrival time) and must reason under uncertainty.
SoC	State of Charge. Battery charge level expressed as a percentage (0-100%). $SoC_{\hat{k}} = \text{ISAC-estimated SoC before plug-in}$. $SoC_k(t) = \text{true SoC at time } t$.
SoH	State of Health. Battery capacity fade factor (0-1). A SoH of 0.85 means the battery can only hold 85% of its original capacity.
PAR	Peak-to-Average Ratio. $P_{\text{peak}} / P_{\text{avg}}$ over the scheduling horizon. Lower PAR = better grid load smoothing. Primary grid-side metric.
TEC	Total Energy Cost. Net electricity cost for all EVs over an episode. Primary user-side metric.
TOU	Time-of-Use pricing. Electricity tariffs that vary by time of day. Peak hours (17-19h) are expensive; off-peak (23-07h) are cheap. The scheduler exploits TOU by charging in off-peak and discharging in peak.
CAISO	California Independent System Operator. Manages the California electricity grid. Their public demand data (2024-05-01 to 2024-05-07) is used as the grid profile in pi6.
Scheduling Lag	Time between EV arrival at a charging station and the first dispatch decision. Reactive systems have high lag (30-90 sec). ISAC-aided proactive systems should have near-zero lag (pre-arrival reservation).
Sensing Gain (delta-J)	Marginal improvement in cumulative reward attributable to ISAC sensing inputs. Computed as: $\text{reward}(\text{ISAC system}) - \text{reward}(\text{same system without sensing inputs in state})$.

wAh model	Weighted Ampere-hour throughput model for battery degradation cost. Degradation penalty = $I * \Delta_{\text{SoC}} * C_{\text{deg_rate}}$, summed over all charge/discharge events.
AEI-V2G	Adaptive Edge Intelligence for Vehicle-to-Grid. The project name and codebase name.

15. Quick Reference — Key Numbers from the Last Run

Metric	Value	Notes
Run date	2026-05-05	Clean rerun after blockchain reset
Run duration	1200 seconds (20 ticks)	Tick = 60 seconds
CAISO profile start	Minute 1020	Evening ramp, high stress window
RSU sense_count (pi5)	78 observations	Repeated per-tick per-EV observations, not unique EVs
Unique EVs routed (pi1)	7	Deduplicated by ev_id
EVs arrived at station (pi2)	2	Still charging at run end; evs_served=0 is correct
Average grid stress (pi6)	0.8246	Above critical threshold 0.8
Average LAVA decision latency (pi1)	0.397 ms	Sub-millisecond on Raspberry Pi
V2G dispatch (pi1)	36.0 kW from tick 15-19	Triggered when stress > 0.8
V2G invitations sent	10	
V2G invitations accepted	5	50% acceptance rate
Energy supplied via V2G	1.0 kWh	Short run — not a performance claim
V2G revenue (pi2)	\$0.6026	Settlement sanity check only
Final blockchain block height	0x98 (152 blocks)	From 0x0 after chain reset
Station C routes received (pi3)	0	KNOWN ISSUE — investigate routing bias

End of Context Document

This document represents the full research and development context for the AEI-V2G project as of May 2026. If any section is unclear or a new decision has been made that changes the direction, update this document before writing new code.

For Claude Code: always read this document fully before making changes. When in doubt, the research objectives in Section 3 take precedence over technical convenience.